MIPS QT6

Host Application

Developer Guide

Version 1.2 • Qt 6.7.1 • Qt Creator 13.0.2

GAA Custom Electronics, LLC
Gordon Anderson • gaa@gaa-ce.com
https://github.com/GordonAnderson/MIPS_QT6

Table of Contents

Table of Contents 2
1. Introduction 4
1.1 Key Capabilities 4
1.2 Coding Conventions 4
2. Architecture Overview 6
2.1 Layered Architecture 6
2.2 Main Window and Source File Split 6
2.3 Communication Stack 6
2.4 ControlPanel Engine and Extracted Widget Classes 7
2.5 TimingGenerator Split 7
2.6 Scripting Runtime 7
3. Build & Setup Instructions 8
3.1 Prerequisites 8
3.2 Cloning the Repository 8
3.3 Opening in Qt Creator (recommended) 8
3.4 Command-Line Build (qmake) 8
3.5 Platform Notes 8
3.6 Version Macro 8
4. Source File Reference 10
4.1 Complete File Inventory 10
4.2 Hardware Module Widgets 13
4.3 ControlPanel Engine 13
4.4 Extracted Widget Classes (Phase 3) 13
4.5 Supporting Infrastructure Classes 14
5. Hardware / MIPS Interface Layer 16
5.1 Physical Connection 16
5.2 MIPS Text Protocol 16
5.3 Command Category Summary 16
5.4 Multi-Controller Support 17
5.5 Compressor Table Syntax 17
6. ECMAScript Scripting API 18
6.1 mips.Command() — UI Control Access 18
6.2 Hardware Communication 18
6.3 UI and Flow Control 18
6.4 Panel Management 19
6.5 ZMQ Interface 19

7. Adding a New Hardware Module 20
7.1 Step-by-Step Checklist 20
7.2 Naming Conventions 20
8. Contributing 21
8.1 Reporting Issues 21
8.2 Branching and Commit Style 21
8.3 Before You Submit 21
8.4 Resources for Contributors 21
9. Glossary 23

1. Introduction

This Developer Guide describes the architecture, build process, source file structure, hardware interface, scripting API, and contribution workflow for the MIPS QT6 host application. It targets software developers who want to understand, modify, extend, or contribute to the codebase.

MIPS (Modular Intelligent Power System) is a scientific instrument control platform by GAA Custom Electronics, LLC. Controllers are used in mass spectrometry and ion mobility instruments to generate high-voltage DC bias, RF drive, traveling-wave, FAIMS, and precision pulse sequences.

The host application communicates with one or more MIPS controllers over USB (CDC serial), Wi-Fi, or Ethernet (TCP/IP). It is written in **C++17 / Qt 6.7.1**, built with Qt Creator 13.0.2 using **qmake** (`MIPS.pro`). A `CMakeLists.txt` is also maintained for IDE compatibility but `MIPS.pro` is the primary build definition.

1.1 Key Capabilities

- Connect via USB, Wi-Fi, or Ethernet to one or multiple MIPS controllers simultaneously
- Tab-based module control — each detected hardware module gets its own dynamically created tab
- Custom control panels — fully graphical instrument interfaces from plain-text .cfg files, no C++ required
- Embedded ECMAScript (JavaScript) scripting for automation, scan loops, and data display
- Real-time ADC acquisition at up to 1 MHz
- Timing generator — clock-accurate pulse sequence editor
- TCP server for remote control from external applications
- ZeroMQ (ZMQ) interface for integration with data acquisition software
- Firmware upload and EEPROM read/write utilities

1.2 Coding Conventions

All source files follow a consistent set of conventions established during the Phase 1-3 refactoring. Contributors must follow these when adding or modifying code.

Language standard	C++17 (Qt 6 minimum requirement)
File header	GAA-style block comment — see any .cpp for the template
Connect syntax	Modern Qt5/6 connect() throughout — NO SIGNAL/SLOT macros (one exception: BlockingQueuedConnection connects to ControlPanel)
Line edit signals	returnPressed not editingFinished for user-entry QLineEdit widgets
Null pointer	nullptr not NULL throughout
Debug output	No qDebug() calls — all removed during refactoring
Dead code	No commented-out dead code

Doc comments	/*! \ brief */ doc comment on every public function (Phase 2 style)
Indentation	4 spaces, no tabs
Naming	camelCase for variables and methods; PascalCase for class names

2. Architecture Overview

2.1 Layered Architecture

Layer	Components
UI / Presentation	MIPS main window, per-module dialog widgets, ControlPanel engine, Plot widget, Console, Help viewer
Application Logic	Program/macro engine, ECMAScript scripting runtime (Qt QML engine), Properties/settings, Firmware updater, TCP server, ZMQ worker
Protocol	Comms class — command framing, response parsing, ring buffer, multi-system routing
Transport	QSerialPort (USB CDC) + QTcpSocket (Wi-Fi/Ethernet), baud-rate negotiation, line buffering
Hardware	MIPS controller firmware (Arduino Due, USB CDC / TCP, MIPS text command set)

2.2 Main Window and Source File Split

The `MIPS` class (`mips.h`) is the QMainWindow that owns the application lifecycle. During Phase 3 refactoring, the original single 2000+ line `mips.cpp` was split into three files, each with a distinct responsibility:

File	Responsibility
mips.cpp	Constructor, destructor, UI/tab management, scripting API, utility methods, menu wiring, settings persistence
connection.cpp	delay(), MIPSsetup(), MIPSconnect(), MIPSsearch(), FindAllMIPSsystems(), FindMIPSandConnect(), MIPSdisconnect(), UpdateSystem()
fileops.cpp	GetFileFromMIPS(), PutFiletoMIPS(), ReadEEPROM(), WriteEEPROM(), ReadARBFLASH(), WriteARBFLASH(), ARBupload()

> **Note:** connection.cpp and fileops.cpp both #include "mips.h" and define methods of the MIPS class. They are not separate classes — they are implementation files for MIPS that were split out for maintainability.

2.3 Communication Stack

Unlike earlier versions of this guide, there is **no separate MIPScomms class**. All protocol and transport logic lives in the `Comms` class (`comms.cpp/.h`). Comms wraps QSerialPort and QTcpSocket, maintains a RingBuffer for received data, and exposes a synchronous send/receive interface to the rest

of the application. The `SettingsDialog` class handles serial port configuration (port name, baud rate, data bits, parity, stop bits, flow control).

2.4 ControlPanel Engine and Extracted Widget Classes

The `ControlPanel` class (`controlpanel.cpp/.h`) was significantly refactored in Phase 3. The original 970-line constructor was split into a `loadConfig()` method, and all 13 embedded widget classes were extracted into their own files. ControlPanel now instantiates these classes by name when parsing the .cfg file.

The extracted classes are listed in full in Section 4.5.

2.5 TimingGenerator Split

The original 1556-line `timinggenerator.cpp` was also split into four files during Phase 3:

File	Class / Content
timinggenerator.cpp/.h	TimingGenerator class + free functions: isTblMode(), DownloadTable(), MakePathUnique()
acquiredata.cpp	AcquireData class — drives an external command-line data acquisition application
eventcontrol.cpp	EventControl class — lightweight widget binding to a single named timing event value
timingcontrol.cpp	TimingControl class — top-level widget wrapping TimingGenerator with Edit/Trigger/Abort buttons and AcquireData

2.6 Scripting Runtime

The application embeds a **Qt QML ECMAScript engine** exposed to control panel scripts via the `mips` object. Scripts are defined in .cfg files between `Script,Name` / `EndScript` delimiters. The `ZmqWorker` class runs the ZeroMQ REQ-REP socket on a background QThread so ZMQ operations do not block the UI. The `TCPserver` class listens on a configurable port and forwards received strings to the ControlPanel command processor.

3. Build & Setup Instructions

3.1 Prerequisites

Qt version	Qt 6.7.1 (Qt 6.5 LTS may work but is untested)
IDE	Qt Creator 13.0.2 recommended
Build system	qmake via MIPS.pro (primary). CMakeLists.txt also maintained.
Compiler (macOS)	Xcode Clang ≥ 14, Apple Silicon (arm64) or Intel
Compiler (Windows)	MSVC 2019/2022 or MinGW 11.2+
Qt modules	Core, Widgets, SerialPort, Network, PrintSupport, QML, SerialBus
Optional: ZMQ	ZeroMQ + cppzmq. macOS: brew install zmq cppzmq. Windows: vcpkg install cppzmq

3.2 Cloning the Repository

```
git clone https://github.com/GordonAnderson/MIPS_QT6.git
cd MIPS_QT6
```

3.3 Opening in Qt Creator (recommended)

1. File → Open File or Project → select MIPS.pro
2. Choose a Qt 6.7 kit for your platform and click Configure Project
3. Press Cmd+B (macOS) or Ctrl+B (Windows/Linux) to build

3.4 Command-Line Build (qmake)

```
qmake MIPS.pro
make -j$(nproc)
```

3.5 Platform Notes

- macOS: built for arm64 (Apple Silicon). See QMAKE_APPLE_DEVICE_ARCHS in MIPS.pro. Serial ports appear as /dev/cu.usbmodemXXXX. No USB driver required.
- Windows: Serial ports appear as COM3, COM4, etc. Install the Arduino Due USB driver before connecting. Copy the MIPS folder to the target PC and run MIPS.exe — no installer required.
- Linux: Add your user to the dialout group: sudo usermod -aG dialout $USER then log out and back in.

3.6 Version Macro

The application version is defined as a compile-time string macro in `MIPS.pro`:

```
DEFINES += APP_VERSION=\"2.23\"
```

This macro is used in the title bar (`setWindowTitle()` in `mips.cpp`) and the About dialog (`aboutdialog.cpp`). When shipping a new release, this is the only line that needs to change for the version to propagate everywhere.

> **Convention:** Always update APP_VERSION in MIPS.pro and add a matching entry to Revision.md before tagging a release on GitHub.

4. Source File Reference

4.1 Complete File Inventory

All source files reside in the repository root. There is no src/ subdirectory.

File(s)	Purpose
main.cpp	Application entry point. Creates QApplication, instantiates MIPS, calls app.exec().
mips.cpp / mips.h / mips.ui	MIPS main window. Constructor, UI/tab management, scripting API invokables, settings, menu wiring.
connection.cpp	MIPS class connection methods — serial/TCP discovery, connect, disconnect, multi-system setup.
fileops.cpp	MIPS class file operations — SD card transfer, EEPROM read/write, ARB FLASH.
comms.cpp / comms.h	All serial and TCP transport. QSerialPort + QTcpSocket wrapper, RingBuffer integration, send/receive.
settingsdialog.cpp/.h/.ui	Serial port configuration dialog. Populates ports via QSerialPortInfo. Exposes Settings struct.
ringbuffer.cpp / ringbuffer.h	Fixed-size (10000 char) circular buffer used by Comms for received serial data.
properties.cpp/.h/.ui	Application preferences dialog. Reads/writes Properties.mips. Exposes paths, flags, TCP list.
aboutdialog.cpp/.h/.ui	About dialog. Shows APP_VERSION, Qt version, build date, copyright, GitHub and Google Drive links.
device.cpp / device.h	Generic device/module discovery and registration.
dcbias.cpp / dcbias.h	DC bias voltage module (up to 32 channels). Setpoint + readback, group-delta mode.
arb.cpp/.h + arbwaveformedit.cpp/.h/.ui	ARB waveform generator module + waveform editor sub-dialog.
faims.cpp / faims.h	FAIMS separation module. RF enable/drive, CV/Bias setpoints, CV parking and scan.
dio.cpp / dio.h	Digital I/O module. 16 TTL outputs + 8 inputs, trigger output controls.
adc.cpp / adc.h	High-speed ADC acquisition up to 1 MHz.
rfdriver.cpp / rfdriver.h	RF driver module. Frequency, amplitude, auto-tune, closed-loop mode.
rfamp.cpp/.h/.ui	RF amplifier / QUAD mass filter module.
twave.cpp / twave.h	Traveling-wave ion guide. Rotation pattern, frequency, pulse voltage, direction.

File(s)	Purpose
filament.cpp / filament.h	Filament current/voltage controller. Ramp rate and cycle control.
compressor.cpp/.h/.ui	Multi-pass ion-mobility compressor timing and mode control.
pse.cpp / pse.h / pse.ui	Power supply enable / interlock module.
psg.cpp / psg.h	Pulse sequence generation module.
modbus.cpp / modbus.h	Modbus RTU pass-through for third-party peripherals on the RS-485 port.
timinggenerator.cpp/.h/.ui	TimingGenerator class + free functions isTblMode(), DownloadTable(), MakePathUnique().
acquiredata.cpp	AcquireData class — drives external command-line acquisition application.
timingcontrol.cpp	TimingControl class — wraps TimingGenerator with Edit/Trigger/Abort + AcquireData.
eventcontrol.cpp	EventControl class — lightweight widget binding to a single named timing event value.
controlpanel.cpp/.h/.ui	ControlPanel engine — parses .cfg files, creates widget instances, runs scripting engine.
tcpserver.cpp / tcpserver.h	TCP server — listens on configurable port, forwards strings to ControlPanel command processor.
zmqworker.cpp / zmqworker.h	ZmqReqRep and ZmqWorker — ZeroMQ REQ-REP socket on a background QThread.
scriptingconsole.cpp/.h/.ui	Standalone scripting console window (Tools → Scripting).
plot.cpp/.h/.ui	Real-time strip-chart plotting widget using QCustomPlot.
qcustomplot.cpp / qcustomplot.h	QCustomPlot library (vendored). Do not modify.
program.cpp / program.h	Macro / sequencing engine.
console.cpp / console.h	Raw serial terminal widget (Terminal tab).
help.cpp/.h/.ui	In-app help viewer — loads *.txt from Qt resources.
cmdlineapp.cpp/.h/.ui	Command-line sub-application execution window.
psviewer.cpp/.h/.ui	Pulse sequence viewer widget.
cdirselectiondlg.cpp/.h	File/directory selection dialog.
Utilities.cpp / Utilities.h	Shared utility functions — tokenisation, numeric helpers, buildDate().
files.qrc	Qt resource file — embeds help texts (MIPShelp, ARBhelp, etc.) and GAACElogo.ico.

File(s)	Purpose
MIPS.pro	qmake project file — primary build definition. Defines APP_VERSION, Qt modules, ZMQ paths.

4.2 Hardware Module Widgets

Each hardware module tab widget follows a consistent pattern:

4. Constructor receives a Comms pointer and system/board index.
5. setupUi(this) loads the .ui layout.
6. A QTimer (pollTimer) fires at the interval set in Properties (default 1 s) and calls readback commands to refresh the display.
7. User edits in spin-boxes or combo-boxes use returnPressed (not editingFinished) to send SET commands.
8. saveSettings() / loadSettings() use QSettings with a unique group name per class.

4.3 ControlPanel Engine

The `ControlPanel` class parses a plain-text `.cfg` file and dynamically instantiates widget objects at run time. Key layout commands:

Command	Effect
`size,x,y`	Sets the panel width and height in pixels.
`image,filename`	Sets a background JPEG/PNG.
`Define,name,value`	Named constant substituted at parse time wherever name appears as a MIPSname argument.
`Tab,name,x,y,w,h,T1,T2,...`	Creates a named tab widget.
`TabSelect,widget,tabname / TabSelectEnd`	Begins/ends a block placed on the named tab.
`GroupBox,title,w,h,x,y[,cw,ch]`	Groups controls. Optional canvas size enables scrolling.
`Subroutine,Name,P1,... / EndSubroutine`	Defines a reusable parameterised layout block.
`Call,Name,V1,...,x,y`	Instantiates a subroutine at (x,y).
`Script,Name / EndScript`	Defines an ECMAScript block.
`TCPserver,port`	Opens a TCP server; external clients send mips.Command() strings.
`SendCommand,MIPSname,cmd`	Sends a MIPS command once at panel load time.
`SkipCount,n`	Sets polling cycles to skip between UI updates.

4.4 Extracted Widget Classes (Phase 3)

These 13 classes were extracted from the original `controlpanel.cpp` during Phase 3. Each lives in its own file pair and follows the same GAA header/doc-comment conventions as all other source files.

Class	Files	Description
Ccontrol	ccontrol.cpp/.h	Generic configurable control: LineEdit, CheckBox, Button, or ComboBox. Hardware-bound or UI-only depending on MIPSname field.
Cpanel	cpanel.cpp/.h	Embedded sub-panel. Creates a child ControlPanel and a button to reveal it. Forward-declares ControlPanel to resolve circular dependency.
TextLabel	textlabel.cpp/.h	Draggable static text label.
Shutdown	shutdown.cpp/.h	System shutdown/enable toggle button. Zeros RF and disables DC bias on press.
SaveLoad	saveload.cpp/.h	Paired Save/Load buttons for panel method files.
CPbutton	cpbutton.cpp/.h	Button that reloads the panel from a specified .cfg file.
DACchannel	dacchannel.cpp/.h	Single DAC output channel widget. Reads GDACV, writes SDACV.
ESI	esi.cpp/.h	Electrospray ionisation high-voltage channel. Setpoint, readback, and enable.
DCBiasGroups	dcbgroups.cpp/.h	DC bias channel group definition widget.
StatusLight	statuslight.cpp/.h	Traffic-light status indicator (GREEN/RED/YELLOW). Also defines LightWidget and TrafficLightWidget helpers.
TextMessage	textmessage.cpp/.h	Dynamic text message display widget.
Table	table.cpp/.h	Editable QTableWidget wrapper. Supports cell scripts and column headers.
Slider	slider.cpp/.h	Horizontal or vertical QSlider widget.

4.5 Supporting Infrastructure Classes

Class (files)	Responsibility
RingBuffer (ringbuffer.*)	Fixed-size 10000-char circular buffer. Used by Comms to stage received serial bytes before line assembly.
SettingsDialog (settingsdialog.*)	Serial port configuration. Exposes a Settings struct (port name, baud, data bits, parity, stop bits, flow control). Re-enumerates ports on every showEvent().
Properties (properties.*)	Persistent application preferences. Reads/writes a CSV-format Properties.mips file. Exposes paths, behaviour flags, TCP host list, and numeric settings as public members.

Class (files)	Responsibility
AboutDialog (aboutdialog.*)	About box. Populated from APP_VERSION macro, QT_VERSION_STR, and __DATE__/__TIME__. Two clickable hyperlinks (GitHub + Google Drive).
TCPserver (tcpserver.*)	QTcpServer subclass. Accepts connections on a configurable port and forwards received command strings to ControlPanel::Command().
ZmqWorker (zmqworker.*)	ZmqReqRep runs a ZeroMQ REQ-REP socket; ZmqWorker moves it onto a background QThread. Exposes REQUEST/REPLY/ERROR/STOP operations to scripts via mips.ZMQ().
ScriptingConsole (scriptingconsole.*)	Standalone scripting window (Tools → Scripting). Allows writing and running ECMAScript outside of a control panel context.
Plot (plot.*)	Real-time strip-chart using QCustomPlot. Exposed to scripts via mips.Command("Tab.PlotName=PlotCommand").
Program (program.*)	Simple line-oriented macro scripts. Supports SET, GET, DELAY, LOOP, WAIT, LABEL.
Console (console.*)	Raw serial terminal widget (Terminal tab).
Help (help.*)	Monospaced plain-text viewer. Loads *.txt from Qt resources or disk. 8-space tab stops, platform-sized Courier font.
Utilities (Utilities.*)	Free functions: tokenise(), toFloat(), clamp(), buildDate() (converts __DATE__ to yyyy-MM-dd).

5. Hardware / MIPS Interface Layer

5.1 Physical Connection

Controller hardware	Arduino Due (Atmel SAM3X8E ARM Cortex-M3)
USB profile	CDC (virtual COM port) — no driver required on macOS
Default baud rate	115200 (configurable in SettingsDialog)
Data format	8N1 — 8 data bits, no parity, 1 stop bit
Flow control	None
Line terminator	\n (0x0A) on both send and receive
Echo mode	Configurable via ECHO command; off by default
TCP/IP	QTcpSocket; enter hostname or IP in the MIPS host name field on the System tab

5.2 MIPS Text Protocol

All communication uses plain ASCII strings terminated by `\n`. The host sends one command and waits for a complete response before sending the next. There is no pipelining. Comms enforces this serialisation via its blocking send/receive methods.

Command pattern: `COMMAND[,arg1[,arg2[,...]]]\n` — uppercase, comma-separated, no spaces.

```
SDCB,1,10.5          ; Set DC bias channel 1 to 10.5 V
GDCBV,1              ; Get DC bias channel 1 actual voltage → "10.5\n"
NAK\n                ; Firmware response for invalid command or out-of-range arg
```

> **Important:** Always call mips.UpdateHalted(true) in scripts before long hardware communication sequences to prevent the background poll timer from interfering.

5.3 Command Category Summary

The full command reference is in MIPScommands.txt (embedded in files.qrc) and displayed in-app via Help → MIPS Commands. Key categories:

Category	Representative Commands
`General / System`	GVER, GNAME/SNAME, GCMDS, RESET, SAVE, ECHO, MUTE, THREADS, SSPND
`DC Bias`	SDCB/GDCB, GDCBV, SDCPWR, GDCBALL/GDCBALLV, SDCBDELTA, SDCBPRO/ADCBPRO, DSTATE/SSTATE
`RF Driver`	SRFFRQ, SRFDRV, SRFVLT, GRFPPVP/N, TUNERFCH, RETUNERFCH, GRFALL
`RF Amplifier / QUAD`	SRFAENA, SRFAFREQ, SRFAMOD, SRFALEV, SRFAMZ, SRFARES, RFAQUPDATE

Category	Representative Commands
ARB / T-Wave	SWFREQ, SWFVRNG, SWFTYP, SWFENA/SWFDIS, SARBCTBL, GTWF/STWF, GTWPV/STWPV, STWSEQ
FAIMS	SFMENA, SFMDRV, SFMCV/GFMCVA, SFMBIAS, SFMCVSTART/END, SFMDUR, SFMTUNE
ADC	SADCRATE/GADCRATE, SADCSAMPS, ADCRBTRIG, ADCRBREAD, ADCRBSUM, ADCABORT
DIO	SDIO/GDIO, SFREQ/GFREQ, SWIDTH/GWIDTH, RPT, MIRROR
Filament	SFLENA/GFLENA, SFLI/GFLI, SFLSV, SFLRT, SFLENAR
ESI (High Voltage)	SHV/GHV, GHVV, SHVENA/SHVDIS, SHVRAMP, SHVGATE
Pulse Table	STBLDAT, STBLCLK, STBLTRG, TBLSTRT, STBLVLT, SMOD, TBLABRT
SD Card / File	DIR, MORE, PUT, GET, SAVEMOD, LOADMOD, SAVEALL, LOADALL, GETEEPROM, PUTEEPROM
Wi-Fi / Ethernet	SHOST/GHOST, SSSID, SPSWD, SWIFIENA, SEIP/GEIP, SEPORT/GEPORT
DAC Module	SDACV/GDACV, SDACMAX, SDACMIN, SDACUN, SDACNM
Pulse Generator	SPGENA, SPGRET, SPGARMT, SPGARML

5.4 Multi-Controller Support

Each connected MIPS controller has its own Comms instance. Controllers are identified by their firmware name (SNAME/GNAME). Control panel Define commands map logical names to physical system names, keeping panels portable across different hardware configurations.

5.5 Compressor Table Syntax

The ARB and T-Wave tabs accept a compact compressor table string (`SARBCTBL` / `STWCTBL`) defining a sequence of waveform states:

N / C / D	Normal cycle / Compression cycle / Delay (hold state, stop clock)
[...]n	Loop: repeat enclosed block n times
V / v	Channel 1 / 2 peak-to-peak voltage (V)
F / O	Waveform frequency / Compression order (O0 = infinite)
c / n	Compress / normal cycle time in ms
S1 / S0	Output switch close / open

Example: `c200v30O5Cv50CN2` — set parameters, compress, adjust, compress, two normal cycles.

6. ECMAScript Scripting API

Control panel `.cfg` files embed ECMAScript (JavaScript) blocks executed by the Qt QML engine. Scripts access hardware and UI through the **mips** object. A script named `StartUp` runs automatically at panel load. A script named `Common` is appended to every other script — define shared utility functions there.

6.1 mips.Command() — UI Control Access

Addressing syntax:

TabName.ControlName	Control inside a tab
TabName.GroupBoxTitle.ControlName	Control inside a GroupBox inside a tab
ControlName	Control directly on the panel (no tab)
TGName.Event.PropertyName	Timing Generator event property
TabName.TableName.Cell,r,c	Table cell (0-based)

```
var v = mips.Command("Tab.ControlName").trim()    // read
mips.Command("Tab.ControlName=newValue")          // write
```

6.2 Hardware Communication

Function	Description
`mips.SendCommand(name, cmd)`	Send cmd to named MIPS system. Always terminate with \n. Returns true if accepted.
`mips.SendMess(name, cmd)`	Send cmd to named system and return response string.
`mips.SendCommand(cmd)`	Single-system form.
`mips.SendMess(cmd)`	Single-system form, returns response.

6.3 UI and Flow Control

Function	Description
`mips.msDelay(ms)`	Pause for ms milliseconds. Required in loops to prevent UI lockup.
`mips.UpdateHalted(bool)`	Stop (true) / resume (false) background UI polling.
`mips.sysUpdating()`	Returns true if a background update cycle is in progress.
`mips.statusMessage(msg)`	Display msg on the status bar.
`mips.popupMessage(msg)`	Modal popup — pauses until dismissed.
`mips.popupYesNoMessage(msg)`	Yes/No dialog. Returns true if Yes.

Function	Description
`mips.popupUserInput(title,msg)`	Text input dialog. Returns entered string.
`mips.SelectFile(mode,title,ext)`	File selection dialog. Returns path string.

> **Caution:** Never write a tight loop without mips.msDelay() inside it. A loop with no blocking call prevents Qt event processing and locks the application.

6.4 Panel Management

Function	Description
`mips.Save(filepath)`	Save all panel settings to file.
`mips.Load(filepath)`	Load settings from file and apply.
`mips.SystemShutdown()`	Zero RF, disable DC bias, save settings.
`mips.SystemEnable()`	Restore from shutdown state.
`mips.Acquire(filepath)`	Start data acquisition (non-blocking).
`mips.isAcquiring()`	Returns true if acquisition is running.

6.5 ZMQ Interface

Command	Description
`mips.ZMQ("BEGIN,address")`	Connect REQ socket to TCP address (e.g. tcp://localhost:5555).
`mips.ZMQ("REQUEST,message")`	Send message asynchronously (non-blocking).
`mips.ZMQ("REPLY")`	Return reply from last REQUEST (empty if not yet arrived).
`mips.ZMQ("ERROR")`	Return error string from last failed REQUEST.
`mips.ZMQ("STOP")`	Close the ZMQ socket.

> **Build requirement:** ZMQ support requires ZeroMQ + cppzmq installed before building. The #define UseZmQ in zmqworker.h gates compilation. MIPS.pro already contains INCLUDEPATH and LIBS entries for both macOS (Homebrew) and Windows (vcpkg).

7. Adding a New Hardware Module

7.1 Step-by-Step Checklist

9. Create <module>.h, <module>.cpp, and optionally <module>.ui in the repo root.
10. Add the GAA-style file header block (see help.h or properties.h for the template).
11. Inherit from QDialog (or QWidget). Include comms.h and store a Comms pointer and system/board index.
12. In the constructor call setupUi(this), set the window title, and connect UI signals to slots using modern connect() syntax — no SIGNAL/SLOT macros.
13. Use returnPressed (not editingFinished) for all user-entry QLineEdit widgets.
14. Add a pollTimer (QTimer) at the Properties update interval and call readback commands in its slot.
15. Implement saveSettings() / loadSettings() using QSettings with a unique group name (use the class name).
16. Add the files to MIPS.pro under SOURCES, HEADERS, and FORMS.
17. In connection.cpp MIPSsetup(), add an AddTab() call and instantiate your class when the firmware reports the new module.
18. Add /*! \brief */ doc comments on every public method.
19. Create <module>help.txt and add it to files.qrc under the "/" prefix.
20. Add a Help menu entry in mips.ui and wire it to the Help viewer.

7.2 Naming Conventions

Source files	All lowercase: mymodule.cpp, mymodule.h, mymodule.ui
Class name	PascalCase: MyModule
MIPS commands	Uppercase, ≤8 chars: SMYMV,ch,val / GMYMV,ch
QSettings group	Class name: settings.beginGroup("MyModule")
Help text	mymodulehelp.txt added to files.qrc as <file>mymodulehelp.txt</file>
Poll interval	Match Properties::UpdateSecs (default 1 s); shorter only if hardware requires it

> **Tip:** Study arb.cpp + arbwaveformedit.cpp as an example of a module that spawns a sub-dialog (waveform editor). Study connection.cpp MIPSsetup() for the AddTab() pattern used by all existing modules.

8. Contributing

MIPS QT6 is in active development and welcomes community partners. Whether you want to add a new hardware module, improve the scripting API, extend the control panel system, or help with cross-platform testing — contributions are very welcome.

8.1 Reporting Issues

Use GitHub Issues at https://github.com/GordonAnderson/MIPS_QT6/issues. Please include:

- Qt version and platform (macOS/Windows/Linux + version number)
- MIPS firmware version — run GVER in the Terminal tab
- Steps to reproduce, expected behaviour, actual behaviour
- Relevant console output or screenshots

8.2 Branching and Commit Style

21. Fork the repository to your GitHub account.
22. Create a feature branch: git checkout -b feature/my-change
23. Use clear commit messages: feat: add MyModule support or fix: correct DCBias readback on multi-system setups
24. Open a Pull Request against main.

8.3 Before You Submit

- Follow all coding conventions in Section 1.2 — particularly modern connect() syntax, returnPressed, nullptr, and the GAA file header block
- Add /*! \brief */ doc comments on every public method you add or modify
- Test with physical MIPS hardware if your change touches Comms, connection.cpp, or any module widget
- Verify QSettings round-trips (save, quit, restart, confirm values restored)
- Check the UI on at least one platform
- Update the relevant *help.txt file if you add or change firmware commands
- Add an entry to Revision.md describing your change

8.4 Resources for Contributors

This guide	Architecture, source file reference, module widget pattern, API
User Manual	MIPS Resources repository — full .cfg syntax and scripting API
Revision.md	Complete version history back to v1.0 (July 2015)
MIPS firmware	github.com/GordonAnderson/MIPS — Arduino Due firmware source

Contact	Gordon Anderson — gaa@gaa-ce.com

9. Glossary

Term	Definition
MIPS	Modular Intelligent Power System — the hardware platform by GAA Custom Electronics, LLC.
Controller	The Arduino Due-based board running MIPS firmware.
Module	A plug-in hardware board (DCBias, ARB, FAIMS, etc.) in the MIPS controller chassis.
Host Application	This Qt 6 desktop application (MIPS_QT6).
CDC	USB Communications Device Class — makes the Arduino Due appear as a serial port.
ACK / NAK	Acknowledge / Negative-Acknowledge — firmware response tokens.
ControlPanel	The runtime-configurable operator interface loaded from a .cfg file.
ECMAScript	JavaScript-compatible scripting language embedded in .cfg files, run by the Qt QML engine.
ARB	Arbitrary Waveform Generator module.
FAIMS	Field Asymmetric Ion Mobility Spectrometry — separation technique and MIPS module.
T-Wave	Travelling Wave — time-varying ion-guide potential, implemented via the ARB module.
ESI	Electrospray Ionisation — high-voltage source for mass spectrometry.
ZMQ / ZeroMQ	Asynchronous messaging library used for external process integration.
TimingControl	Top-level widget wrapping TimingGenerator with Edit/Trigger/Abort and AcquireData.
AcquireData	Class that drives an external command-line data acquisition application.
RingBuffer	Fixed-size 10000-char circular buffer used by Comms for received serial data.
APP_VERSION	Compile-time string macro defined in MIPS.pro. Used in title bar and About dialog.
qmake / MIPS.pro	The primary build system for this project. CMakeLists.txt is also maintained.
Ccontrol	Generic configurable control widget — the most versatile widget in the .cfg system.
Compressor Table	Compact string syntax defining ARB/T-Wave waveform state sequences for the multi-pass ion compressor.